

# **ECEN 4632 - Final Project**

## **Audio Processing**

**Dr. Talles Santos**

**Oliver Boyd, Carlos Rodriguez, Viktor Woldruff**

**Contents**

|                            |                                     |
|----------------------------|-------------------------------------|
| <b>Introduction:</b> ..... | 3                                   |
| <b>Materials:</b> .....    | 3                                   |
| <b>Methodology:</b> .....  | 3                                   |
| <b>Results:</b> .....      | 5                                   |
| <b>Limitatons:</b> .....   | <b>Error! Bookmark not defined.</b> |
| <b>Conclusions:</b> .....  | 7                                   |
| <b>References:</b> .....   | 7                                   |
| <b>Appendix:</b> .....     | 8                                   |

## Introduction:

Audio manipulation has a wide range of real-life applications, from protecting the identity of witnesses to children's toys. It involves various techniques that modify the frequency of audio signals, either lowering or raising the pitch depending on the desired outcome. This project focuses on utilizing the Analog Discovery 2 (AD2) and MATLAB to perform real-time signal acquisition, filtering, pitch shifting, and audio playback. Specifically, the replication of the "Pbooo Dancing Talking Cactus Mimicking Toy," as seen on Amazon.com.

## Materials:

- Diligent Analog Discovery 2
- MATLAB
  - Data Acquisition Toolbox
- Amplification Circuit
  - 4x - 10uF Capacitor
  - 1x - 0.01uF Capacitor
  - 1x - Microphone
  - 1x - 2.2kOhm Resistor
  - 2x - 1kOhm Resistor
  - 1x - 22kOhm Resistor
  - 1x - LF411 Operational Amplifier
  - 1x - Series DC Power Supply

## Methodology:

### 1. Overview:

To explore the potential of real-time audio manipulation, a comprehensive methodology was developed. This project focuses on utilizing the Analog Discovery 2 (AD2) alongside MATLAB for signal acquisition, processing, and playback to replicate the behavior of the "Pbooo Dancing Talking Cactus Mimicking Toy" available on Amazon.com. By leveraging the AD2, the system captures audio signals in real time, applies filtering to isolate specific frequency ranges, and implements pitch-shifting algorithms to alter the signal. The processed audio is then replayed, mimicking the toy's functionality.

### 2. Data Acquisition

To generate an audio signal large enough to be measured by the AD2, the microphone's output needed to be amplified using the analog amplification circuit shown in Figure 1. This circuit was designed to provide a gain of 22 ( $R_6/R_5$ ), producing a voltage signal that is sufficiently large for measurement but remains within the AD2's  $\pm 25V$  input range. Once a usable signal was generated, the AD2 acquired data from the microphone using MATLAB's Data Acquisition Toolbox, which interfaces with the AD2 hardware. The system begins by initializing the Diligent Analog Discovery device and configuring it for single-channel voltage input. The sampling rate is set at 44.1 kHz [1] to ensure high-resolution data capture, while a 30-microsecond sampling window provides a

consistent acquisition period for signal thresholding. Although the AD2's native software, Waveforms, supports continuous data acquisition, this feature is unavailable in MATLAB's Data Acquisition Toolbox. To address this limitation, a primary while loop enables continuous data reading from the configured input channel.

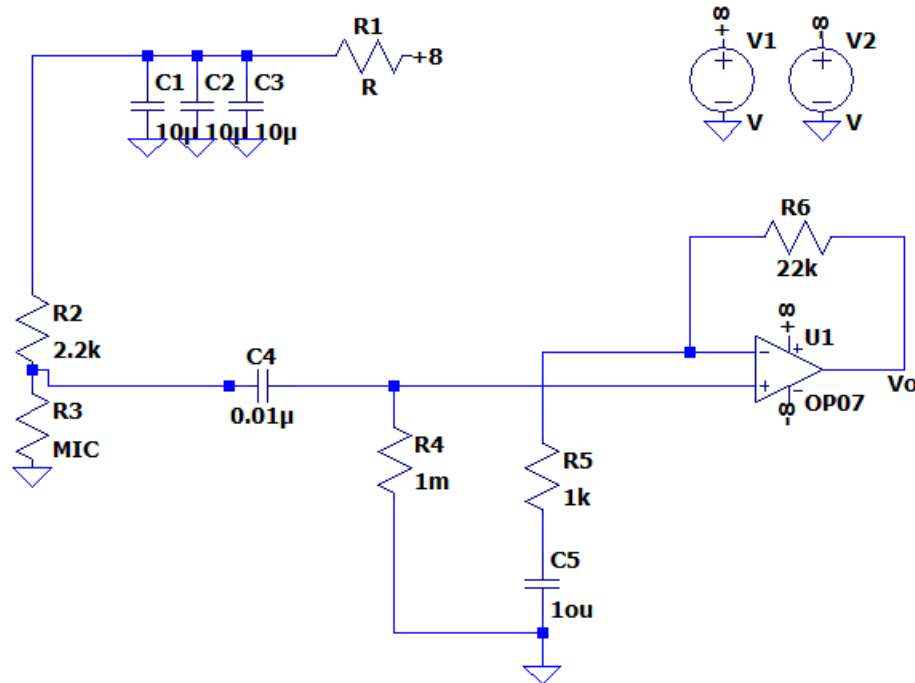


Figure 1. The schematic of the amplifier circuit used to obtain audio signals.

Each loop iteration calculates the mean voltage of the acquired samples to detect significant audio activity. If the voltage exceeds a predefined threshold of 0.3V, the system identifies the presence of audio input and initiates a secondary acquisition phase, capturing a 3-second sample and storing it in an array. This duration accommodates most typical spoken phrases. Furthermore, if the absolute mean voltage of the captured signal remains above a secondary threshold of 0.045V, the system assumes sustained audio input and appends subsequent voltage data segments to the signal array for continued recording. This dynamic recording continues until the signal falls below the threshold, signifying the absence of significant audio input.

### 3. Data Processing in MATLAB - Filtering, Shift Pitch

To reduce high-frequency noise, the audio signal obtained by the AD2 is processed using a 12-coefficient low-pass Butterworth filter with a cutoff frequency of 4 kHz. This cutoff frequency was chosen because the human voice typically falls within the 85 Hz to 255 Hz frequency range[2], and the average upper limit of human hearing is around 15 kHz[3]. This allows the device to process the full range of human voice frequencies, as well as most music frequencies. The filtered signal's pitch is then shifted using MATLAB's `shiftpitch()` function, which adjusts the audio by the desired number of semitones. This function applies time-compression via interpolation and then resamples the signal to its original duration at the new frequency, resulting in a pitch-shifted audio output. After all

processing has been completed, the data is written to the AD2 to be sent from its DAC to the speaker for playback.

### Results:

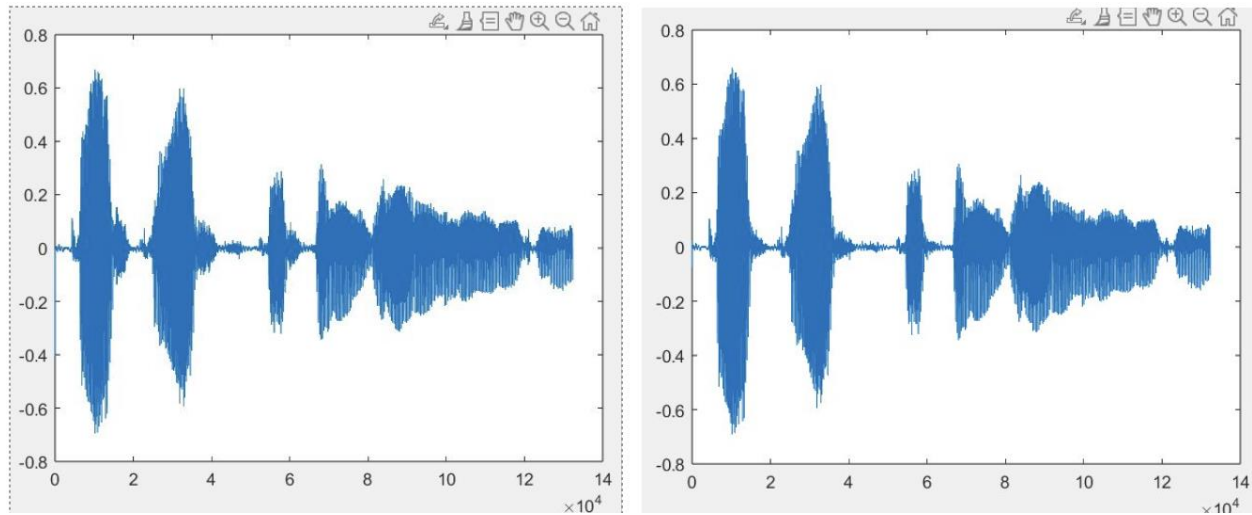


Figure 2. MATLAB results of the audio signal before applying filtering and audio pitching shown on the left and after applying the Butterworth low-pass filter on the right

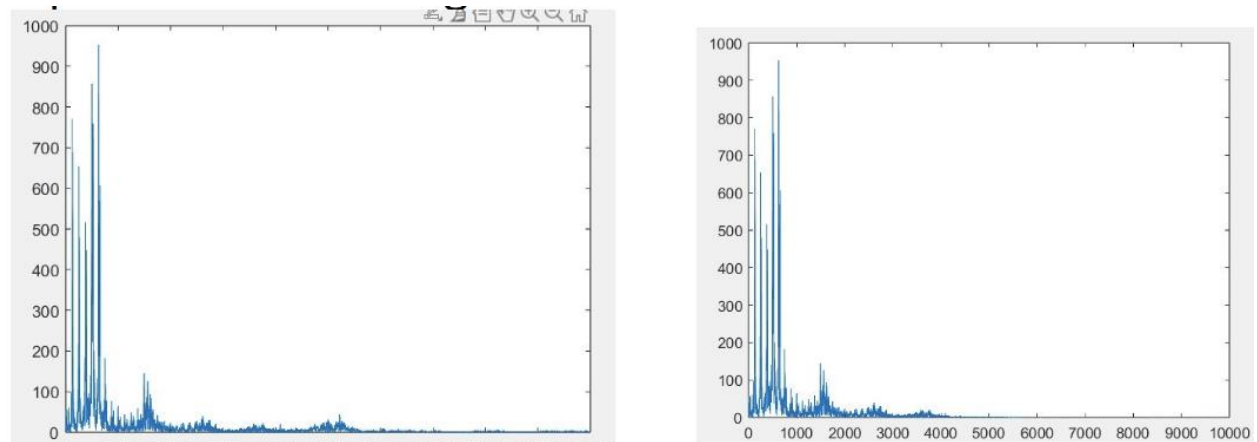


Figure 3. MATLAB results of the FFT of the audio signal without filtering and audio pitching shown on the left and after applying the Butterworth low-pass filter on the right

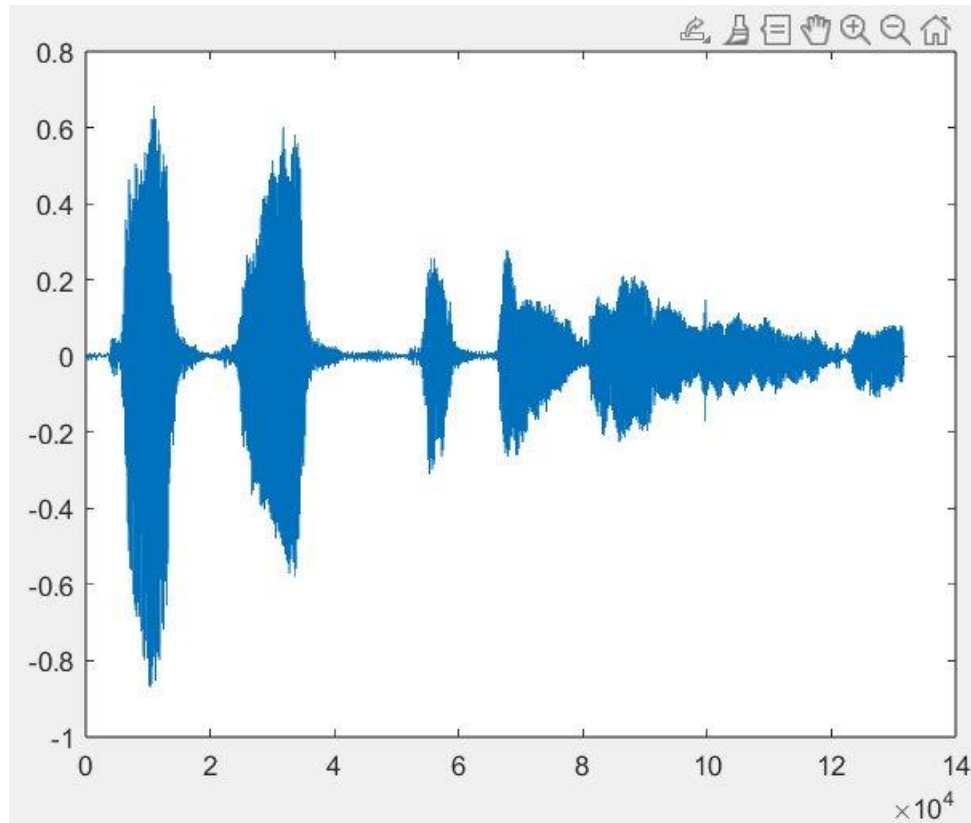


Figure 4. MATLAB results of the audio signal after applying the Butterworth low-pass filter and pitch shifting

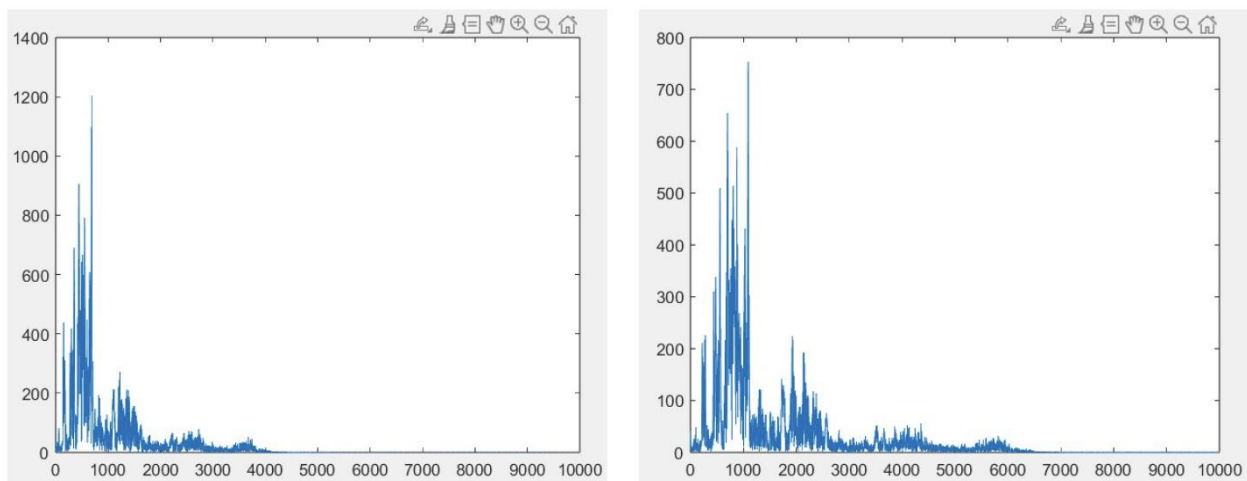


Figure 5. MATLAB results of the FFT of the audio signal with the Butterworth low-pass filter applied before the pitch shifting on the left and the FFT of the audio signal after the pitch shift was applied

The implementation of the real-time filtering and pitch-shifting process successfully modified the acquired audio signal in accordance with the desired parameters. Initially, the audio signal that was obtained from the microphone circuit had noticeable background noise, which was effectively reduced through the application of the 4 kHz cut-off frequency Butterworth-lowpass filter. As shown in Figure 2 and Figure 3, the signals clarity improved after applying the filter, with the high-frequency noise being reduced leaving a cleaner signal for processing and displaying through a speaker.

Following the filtering, pitch-shifting was applied using MATLAB's `shiftpitch()` function. The audio's pitch was successfully altered the audio signal up 8 semitones which is the equivalent to shifting up a full octave, and as shown in Figure 4, the waveform reflects this change. Figure 5 further shows the frequency content after both filtering and pitch-shifting, showing the smooth transition of the audio signal.

The pitched audio was successfully displayed on the speaker, though some limitations were observed. After the audio was captured, a slight delay occurred during the processing and outputting of the signal. This delay is believed to be due to processing and outputting data in MATLAB, which, in combination with an AD2, is not the most optimized method. While this delay is noticeable, it does not significantly affect the system's overall performance.

Another limitation involved a brief cut-off of the initial seconds of the modified audio, caused by data loss during the capture process. This issue arises because the AD2 cannot perform continuous data acquisition in MATLAB. Because of this there will be a loss in the capturing of the data which was reduced by the method of capturing a 3-second sample to start acquisition. This limitation can be bypassed by the user saying the word "testing" first, followed by the audio that is desired to be repeated back with modification. This initial segment provides enough buffer time to account for the delay, ensuring the rest of the desired audio could be processed and played back without interruption. Once this accommodation is met, the system correctly outputs modified audio, mimicking the behavior of a phrase repeating toy like the "Pbooo Dancing Talking Cactus Mimicking Toy."

## **Conclusions:**

This project successfully demonstrated the ability to perform "real time" audio acquisition, filtering, pitch shifting, and playback using the Analog Discovery 2 and MATLAB. By integrating hardware and software, the system replicated the functionality of a consumer-grade audio-processing toy, such as the "Pbooo Dancing Talking Cactus Mimicking Toy."

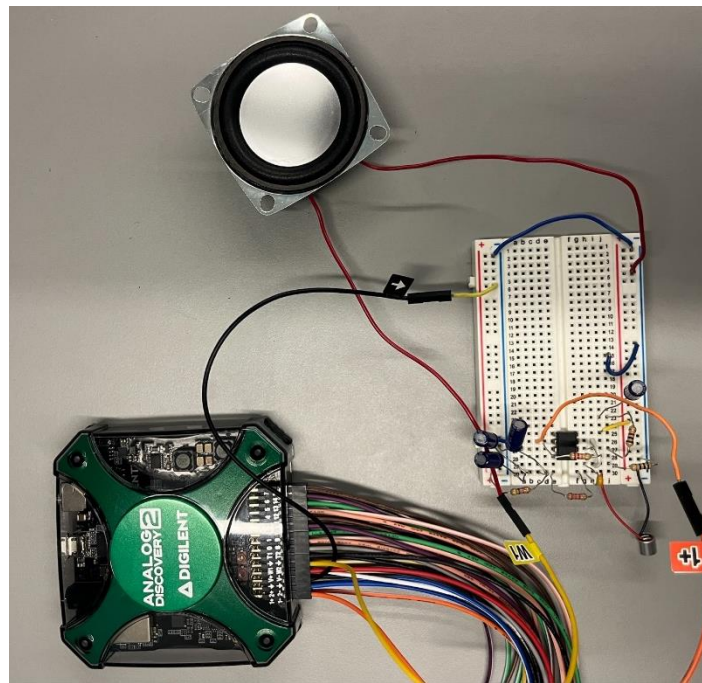
The implementation of a Butterworth low-pass filter effectively reduced high-frequency noise, enhancing the clarity of the captured audio signal. Pitch shifting was achieved using MATLAB's `shiftpitch()` function, allowing precise modulation of the signal by a full octave, as reflected in the visualized waveforms and frequency spectra. While the processed audio was successfully replayed through a speaker, the project identified two key limitations: a slight delay in processing and playback, and an initial data loss during signal capture due to the AD2's non-continuous acquisition limitations in MATLAB.

Despite these challenges, the workaround of preemptive speech: "testing," allowed for seamless playback of the desired audio. Overall, the system proved capable of modifying audio signals in real time and effectively mimicked the target device's behavior, showcasing the potential of combining MATLAB and AD2 for interactive audio-processing applications. Future optimizations could focus on reducing delays and enabling continuous data acquisition to improve the user experience further.

### References:

- [1] A.-M. Lessons, "What is the best audio sample rate? 44.1, 48, 96? Sample rate explained," [mixinglessons.com](https://www.mixinglessons.com/sample-rate/). Accessed: Dec. 04, 2024. [Online]. Available: <https://www.mixinglessons.com/sample-rate/>
- [2] "Frequency Ranges in Voice Over Recording - The DIY Voice Over Website Template Builder - VoiceActor.com." Accessed: Dec. 04, 2024. [Online]. Available: <https://voiceactor.com/articles/frequency-ranges-in-voice-over-recording>
- [3] D. Purves *et al.*, "The Audible Spectrum," in *Neuroscience*. 2nd edition, Sinauer Associates, 2001. Accessed: Dec. 04, 2024. [Online]. Available: <https://www.ncbi.nlm.nih.gov/books/NBK10924/>

### Appendix:



Appendix Figure 1. Circuit used for the project.





Appendix Figure 2. Picture of the final design of the project.

[https://drive.google.com/file/d/1AdSd5FlpLD1e0HxiCCFKafVZweWSQNrx/view?usp=drive\\_link](https://drive.google.com/file/d/1AdSd5FlpLD1e0HxiCCFKafVZweWSQNrx/view?usp=drive_link)

Link of the project working.

---

```

while(1)

    close all; clear all; clc; daqreset;

    daqlist("digilent");
    dq = daq("digilent");
    addinput(dq, "AD1", "1", "Voltage");

    ch_in = dq.Channels(1);
    ch_in.Name = "AD1_1_in";
    rate = 44.1e3;
    dq.Rate = rate;

    secs = 3e-5;

    v1 = [];
    t1 = [];

    while true
        data = read(dq, seconds(secs));
        voltageData = data.AD1_1_in;
        meanVoltage = mean(voltageData);

        if abs(meanVoltage) > 0.3 && meanVoltage < 5
            v1 = [v1;voltageData];
            disp("Threshold Met");
            data = read(dq, seconds(3));
            triggeredVoltage = data.AD1_1_in;
            triggeredTime = data.Time;

            while mean(abs(triggeredVoltage(1:44100))) > 0.045
                disp("Recording");
                v1 = [v1;triggeredVoltage];

                data = read(dq, seconds(1));
                triggeredVoltage = data.AD1_1_in;
                triggeredTime = data.Time;
            end
            disp("Break");
            break;
        end
        disp(num2str(meanVoltage));
    end

    if length(v1)>5
        y = v1;
        y = y';
        % y = max(min(data, 5), -5);
        % y = table2array(y);
        fc = 4000;
        [b, a] = butter(12,fc/(rate/2));
        y = filter(b,a,y);
    end
end

```

---

---

```
y1 = shiftPitch(y,8);
y2 = shiftPitch(y,-4);

daqreset;
pause(1);

daqlist("digilent");
dq = daq("digilent");
addoutput(dq, "AD1", "1", "Voltage");
ch_out = dq.Channels(1);
ch_out(1).Name = "AD1_1_out";
dq.Rate = 44.1e3;
write(dq,y1);

% t = linspace(0,length(y),length(y));
% ffty = fft(y,rate);
% figure;
% plot(abs(ffty));
% xlim([0,10000])
%
% ffty1 = fft(y1,rate);
% figure;
% plot(abs(ffty1));
% xlim([0,10000])

end
end
```

Error using BoydRodriguezWoldruff\_ECEN4632\_Fall2024\_FinalProject (line 6)  
Specify an operational vendor. Use daqvendorlist for a list of vendors.

*Published with MATLAB® R2024a*